

Relatório – Chat via UDP

Nina Lira Henriques de Araújo

A aplicação implementa, sobre um canal UDP não confiável (com 40% de perda simulada de pacotes), um mecanismo de confirmação de entrega (Application-level ACK) inteiramente na camada de aplicação, seguindo o protocolo textual delimitado por '|' definido no enunciado: MSG| para mensagens e DELIVERED| para recibos.

1. Chat_receiver

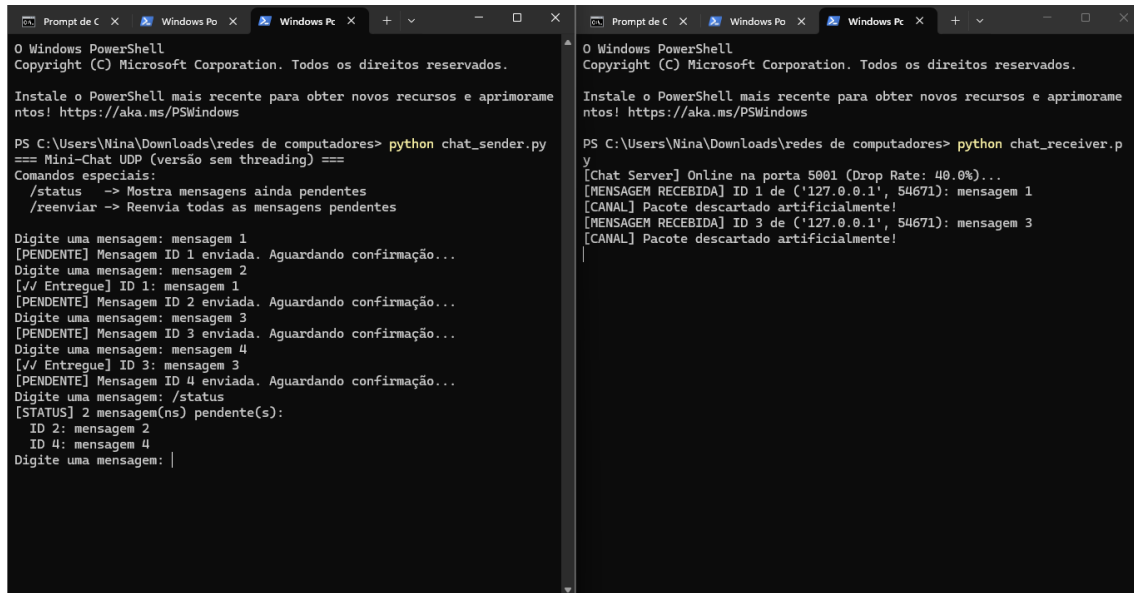
O servidor escuta continuamente na porta 5001. Cada datagrama recebido passa primeiro pelo filtro de descarte artificial (`random.random() < DROP_RATE`); se descartado, o pacote é simplesmente ignorado, simulando a perda que ocorreria em roteadores intermediários. Se o pacote sobrevive, a mensagem é separada pelo delimitador '|' (com limite de 2 splits, para não quebrar o conteúdo caso ele próprio contenha o caractere '|'), o tipo MSG é validado, o ID e o texto são extraídos, exibidos no terminal, e um recibo DELIVERED| é enviado de volta ao endereço de origem do pacote (obtido diretamente do `recvfrom`, sem precisar de configuração adicional).

2. Chat_sender

O cliente mantém um dicionário `pending_messages` (ID → texto) com todas as mensagens que ainda não foram confirmadas. Cada mensagem digitada recebe um ID sequencial, é registrada nesse dicionário como pendente e é enviada ao servidor. Um mecanismo de leitura não-bloqueante do socket verifica, a cada iteração do laço principal, se chegaram recibos DELIVERED; quando um recibo é reconhecido, a mensagem correspondente é removida de `pending_messages` e uma confirmação visual [✓✓ Entregue] é exibida. O comando `/status` lista as mensagens que ainda restam em `pending_messages`, e o comando `/reenviar` reenvia, com o mesmo ID original, todas as mensagens ainda pendentes — permitindo que o servidor as reconheça e confirme normalmente, já que o ID não muda entre tentativas.

Observação de implementação: inicialmente a leitura dos recibos foi feita em uma thread separada rodando em segundo plano. Durante os testes, essa abordagem apresentou instabilidade no ambiente Windows utilizado (os recibos não eram processados mesmo com o firewall desativado). Para garantir robustez e previsibilidade, a solução final substitui a thread por uma varredura não-bloqueante do socket (`socket.settimeout(0)`) executada a cada iteração do laço principal do cliente, eliminando a dependência de concorrência entre threads para esta funcionalidade.

Na captura abaixo, quatro mensagens são enviadas pelo cliente (esquerda). O servidor (direita) recebe as mensagens 1 e 3, mas descarta artificialmente os pacotes correspondentes aos IDs 2 e 4 (simulação de perda de canal). No cliente, o comando /status é utilizado antes que os IDs 2 e 4 sejam confirmados, mostrando corretamente as duas mensagens ainda pendentes.



```
O Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimorame
ntos! https://aka.ms/PSWindows

PS C:\Users\Nina\Downloads\redes de computadores> python chat_sender.py
== Mini-Chat UDP (versão sem threading) ==
Comandos especiais:
/status -> Mostra mensagens ainda pendentes
/reenviar -> Reenvia todas as mensagens pendentes

Digite uma mensagem: mensagem 1
[PENDENTE] Mensagem ID 1 enviada. Aguardando confirmação...
Digite uma mensagem: mensagem 2
[✓✓ Entregue] ID 1: mensagem 1
[PENDENTE] Mensagem ID 2 enviada. Aguardando confirmação...
Digite uma mensagem: mensagem 3
[PENDENTE] Mensagem ID 3 enviada. Aguardando confirmação...
Digite uma mensagem: mensagem 4
[✓✓ Entregue] ID 3: mensagem 3
[PENDENTE] Mensagem ID 4 enviada. Aguardando confirmação...
Digite uma mensagem: /status
[STATUS] 2 mensagem(ns) pendente(s):
    ID 2: mensagem 2
    ID 4: mensagem 4
Digite uma mensagem: |

O Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimorame
ntos! https://aka.ms/PSWindows

PS C:\Users\Nina\Downloads\redes de computadores> python chat_receiver.p
y
[Chat Server] Online na porta 5001 (Drop Rate: 40.0%)...
[MENSAGEM RECEBIDA] ID 1 de ('127.0.0.1', 54671): mensagem 1
[CANAL] Pacote descartado artificialmente!
[MENSAGEM RECEBIDA] ID 3 de ('127.0.0.1', 54671): mensagem 3
[CANAL] Pacote descartado artificialmente!
```

Em seguida, o comando /reenviar é utilizado para reenviar as mensagens dos IDs 2 e 4, ainda pendentes. Desta vez, os pacotes não são descartados pelo canal: o servidor recebe ambas as mensagens (mantendo os mesmos IDs originais) e devolve os recibos DELIVERED correspondentes, que o cliente processa confirmando a entrega das duas mensagens com o indicador [✓✓ Entregue].

```
O Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimorame
ntos! https://aka.ms/PSWindows

PS C:\Users\Nina\Downloads\redes de computadores> python chat_sender.py
=== Mini-Chat UDP (versão sem threading) ===
Comandos especiais:
  /status -> Mostra mensagens ainda pendentes
  /reenviar -> Reenvia todas as mensagens pendentes

Digite uma mensagem: mensagem 1
[PENDENTE] Mensagem ID 1 enviada. Aguardando confirmação...
Digite uma mensagem: mensagem 2
[✓✓ Entregue] ID 1: mensagem 1
[PENDENTE] Mensagem ID 2 enviada. Aguardando confirmação...
Digite uma mensagem: mensagem 3
[PENDENTE] Mensagem ID 3 enviada. Aguardando confirmação...
Digite uma mensagem: mensagem 4
[✓✓ Entregue] ID 3: mensagem 3
[PENDENTE] Mensagem ID 4 enviada. Aguardando confirmação...
Digite uma mensagem: /status
[STATUS] 2 mensagem(ns) pendente(s):
  ID 2: mensagem 2
  ID 4: mensagem 4
Digite uma mensagem: /reenviar
[REENVIADO] ID 2: mensagem 2
[REENVIADO] ID 4: mensagem 4
[✓✓ Entregue] ID 2: mensagem 2
[✓✓ Entregue] ID 4: mensagem 4
Digite uma mensagem: |

O Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimorame
ntos! https://aka.ms/PSWindows

PS C:\Users\Nina\Downloads\redes de computadores> python chat_receiver.py
[Chat Server] Online na porta 5001 (Drop Rate: 40.0%)...
[MENSAGEM RECEBIDA] ID 1 de ('127.0.0.1', 54671): mensagem 1
[CANAL] Pacote descartado artificialmente!
[MENSAGEM RECEBIDA] ID 3 de ('127.0.0.1', 54671): mensagem 3
[CANAL] Pacote descartado artificialmente!
[MENSAGEM RECEBIDA] ID 2 de ('127.0.0.1', 54671): mensagem 2
[MENSAGEM RECEBIDA] ID 4 de ('127.0.0.1', 54671): mensagem 4
|
```

Discussão: controle na aplicação vs. confiabilidade nativa do TCP

O TCP resolve o problema de perda de pacotes de forma totalmente transparente à aplicação: a própria pilha de protocolos do sistema operacional cuida de números de sequência, temporizadores de retransmissão, reconhecimento (ACKs) e reordenação de segmentos, entregando à aplicação um fluxo de bytes confiável e ordenado. O custo dessa confiabilidade é o estabelecimento prévio de conexão (three-way handshake), maior overhead de cabeçalho e controle, e menor controle fino sobre o que e quando retransmitir.

Ao implementar esse controle manualmente na camada de aplicação sobre UDP, como feito nesta atividade, a aplicação ganha flexibilidade: pode decidir, por exemplo, ignorar mensagens antigas, agrupar reenvios sob demanda do usuário (/reenviar) em vez de retransmissão automática imediata, ou aplicar políticas próprias de tempo de espera. Em contrapartida, toda a complexidade de identificar perdas, gerar identificadores únicos, esperar confirmações e reenviar fica sob responsabilidade do próprio programador — não há garantia de ordem de entrega nem de que os recibos, por sua vez, não se percam (um DELIVERED também poderia ser descartado pelo canal, cenário não tratado nesta versão simplificada). Esse é exatamente o tipo de escolha de projeto (controle e simplicidade de protocolo vs. confiabilidade automática) que motiva o uso de UDP com ACKs de aplicação em sistemas de mensageria em tempo real, priorizando baixa latência em detrimento de garantias absolutas de entrega.